

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia
Curso de Ciência da Computação
Prof. Fabricio Bortoluzzi

Nome do estudante

Gustavo Henrique Stahl Müller

E-mail (utilizar email @edu.univali.br)

gustavomuller@edu.univali.br

[X] Declaro que o conteúdo deste trabalho é integralmente de minha autoria com apoio do material didático indicado pelo professor da disciplina devidamente referenciado no corpo do documento.

Observações:

1. Siga cuidadosamente as instruções de cada tarefa
2. Preserve o formato original desse documento. Não faça alterações de estilo, espaçamento, etc.
3. Trabalhe de forma individual. Não compartilhe seus resultados para evitar detecção de plágio.
4. Fique à vontade para trocar ideias com seus pares, inclusive no grupo não-oficial, informal e opcional do Whatsapp, neste [link](#).

Este trabalho será usado para composição das duas primeiras notas (M1) da disciplina. A primeira nota se dará pela Tarefa 1. A segunda nota se dará pela qualidade geral das Tarefas 2 e 3 descritas no corpo do documento.

Esta atividade deve ser submetida até domingo, dia 02 de outubro, às 23:00 horas, impreterivelmente, exclusivamente na Intranet/Material Didático na seção descrita como Atividade 2.

Tarefa 1

Em nossas aulas, empregamos a seguinte definição para Sistemas Distribuídos - SD:

“Coleção de elementos computacionais autônomos que parecem um único sistema coerente para seus usuários”

Os slides da matéria, disponíveis [aqui](#), detalham o assunto em diversos aspectos. Em especial, temos muito para aprender sobre SD no aspecto de transparência:

1. **Acesso:** esconder diferenças na representação de dados e como um objeto é acessado;
2. **Localização:** esconder onde um objeto está localizado;
3. **Relocação:** esconder que um objeto pode ser movido para outro local enquanto é utilizado;
4. **Migração:** esconder que um objeto pode mover-se para outro local;
5. **Replicação:** esconder que um objeto é replicado;
6. **Concorrência:** esconder que um objeto pode ser compartilhado por diversos usuários; e
7. **Falha:** esconder que um objeto pode sofrer falha e deve ser restaurado.

A Tarefa 1 requer que você escreva um texto de sua autoria, que deve ser inspirado na leitura de um dos dois livros textos recomendados explicando cada um dos sete atributos de transparência.

Você vai encontrar explicações tanto nos slides, quanto nos livros, quanto nas próprias aulas. Porém, aqui o desafio refere-se a sua capacidade de escrever um ensaio autorial. Certamente espero que você referencie o livro utilizado no fim do seu texto, porém, diferentemente de um artigo científico, aqui basta você sintetizar com seu vocabulário. Seja preciso, faça bom uso do idioma português e evite usar exemplos como forma de explicar suas ideias. O que importa mesmo é ficar dentro do assunto e escrever como se estivesse me explicando para eu aprender esse assunto a partir do zero.

Não há limite mínimo ou máximo de texto, porém sugiro tentar usar 1 (uma) página deste modelo para cada um dos sete tópicos listados.

Eu vou atentar para sua capacidade de síntese, sua clareza, fluência do texto e claro, originalidade, no sentido que se deve evitar copiar ou compartilhar sua produção com seus colegas evitando detecção de plágio que poderá requerer investigação minuciosa do professor.

Tópico 1 – Transparência de Acesso:

A transparência de acesso se refere à abstração dos mecanismos de acesso aos dados presentes em diferentes nodos do sistema. Há varios fatores que poderiam causar incompatibilidade entre representações de dados em diferentes nodos. Por exemplo:

- Diferentes sistemas operacionais usados por nodos distintos podem possuir sistemas de arquivos diferentes. Consequentemente, arquivos são representados de maneiras diferentes em disco.
- Diferentes nodos também podem possuir diferentes tecnologias de armazenamento, como disco rígidos ou SSDs. Normalmente a transparência de acesso entre diferentes tecnologias de armazenamento é disponibilizada pelo sistema operacional, geralmente com o uso de um sistema de arquivos virtual (VFS).
- Por último, é possível que diferentes partes de um mesmo objeto estejam salvas em diferentes partes do sistema.

Um sistema distribuído que possui transparência de acesso esconde essas diferenças e apresenta uma interface de acesso à dados uniforme ao usuário. Ou seja, objetos podem ser acessados pelos menos mecanismos independentemente de onde estão salvos no sistema.

Um exemplo disso é o sistema de arquivos do Network File System (NFS). O NFS permite a criação de arquivos e diretórios virtuais que são compartilhados entre máquinas de uma rede do mesmo modo que o usuário cria arquivos e diretórios no seu computador.

Tópico 2 – Transparência de Localização:

A transparência de localização se refere à abstração da localização de objetos no sistema. A vantagem em esconder a localização real dos objetos é a maior flexibilidade, visto que mudanças de localização não afetam a maneira que o usuário opera o sistema.

Um exemplo de transparência de localização é a URL (Uniform Resource Locator). Sem uma URL, o usuário precisa do endereço IP do servidor que deseja se comunicar. Porém, endereços IP devem conter um prefixo de sub-rede, que é diferente para localizações distintas. Logo, caso um servidor mude de localização física, sua sub-rede muda, fazendo com que o seu endereço IP mude também. Para que a comunicação continue, todos os hosts que se comunicam com o servidor devem estar cientes da mudança de IP. Isso é um exemplo de falta de transparência de localização.

A transparência pode ser assegurada através do uso de nomes lógicos que independem de localização. No caso do URL, o nome lógico é transformado em um endereço IP através do protocolo DNS. Logo, quando um servidor muda de localização física, somente o serviço DNS precisa estar ciente da mudança.

Outra vantagem da transparência de localização é que como o usuário não sabe onde os serviços que ele deseja usar realmente estão implementados, é possível usar um balanceador de carga que encaminha a requisição de um usuário para nodos menos ativos ou que possuem mais memória disponível.

Tópico 3 – Transparência de Relocação:

A transparência de relocação se refere à abstração de quando um certo recurso passou a estar disponível em um certo nodo do sistema. Por exemplo, a URL “<http://www.prenhall.com/index.html>” não indica se o recurso “index.html” sempre esteve disponível no domínio “www.prenhall.com” ou se ela acabou de ser movida para lá.

Tópico 4 – Transparência de Migração:

A transparência de migração se refere à abstração da movimentação dos processos e recursos iniciados pelo usuário, com o objetivo de manter todas as comunicações e operações em andamento.

Um exemplo de transparência de migração seria a migração de um serviço de vídeo por *streaming* de um nodo para outro sem que o usuário soubesse que tal migração aconteceu.

Um exemplo de falta de transparência de migração seria marcar um recurso (como um vídeo, imagem, etc) como “offline” enquanto ele está sendo movido de um nodo do sistema para outro.

Tópico 5 – Transparência de Replicação:

A transparência de replicação se refere à abstração do fato da existência de múltiplas réplicas de um mesmo recurso. Existem várias vantagens em criar réplicas do mesmo objeto em locais diferentes de um sistema distribuído, como:

- Confiabilidade, que age como um modo de proteção contra falhas, já que caso um nodo que contenha uma réplica de um objeto seja desabilitado ou deixe de funcionar, outro nodo que possua uma réplica do mesmo objeto ainda pode ser usado para acessar o objeto.
- Redundância, pois réplicas também atuam como proteção contra corrupção de dados. Por exemplo, caso existam cinco réplicas de um objeto, se uma delas for corrompida de alguma maneira, ainda é possível considerar o valor das quatro outras réplicas.
- Desempenho, já que diferentes réplicas do mesmo dado podem ser colocadas em posições geográficas mais próximas de usuários, reduzindo a latência;

Um usuário que interage com um sistema distribuído que possui transparência de replicação não sabe quantas réplicas de um mesmo objeto existem. Caso uma das réplicas seja alterada, todas as réplicas devem ser alteradas igualmente de maneira atômica (caso contrário, é possível que o sistema forneça informações desatualizadas).

Um exemplo de transparência de replicação é o uso de cache de páginas web por navegadores. Quando um navegador recebe uma resposta HTTP contendo uma página, ela é salva em cache localmente no computador do usuário. Caso no futuro o usuário faça uma requisição pela mesma página, a página salva em cache é usada, tirando a necessidade de realizar uma requisição HTTP e consequentemente reduzindo a latência. O usuário não sabe se ele está lendo uma página recém requisitada ou uma página previamente salva.

Caso a página tenha sido alterada desde o seu salvamento em cache, é possível que a página mostrada esteja desatualizada, logo, deve existir algum modo de atualizar as réplicas da página quando a página original é alterada.

Tópico 6 – Transparência de Concorrência:

A transparência de concorrência se refere à abstração do uso concorrente de um recurso por diferentes usuários. Ou seja, o sistema fornece a ilusão ao usuário de que somente ele está usando o objeto em questão.

Caso diferentes usuários atualizem um mesmo objeto ao mesmo tempo, é necessário que a execução concorrente das transações não acabe deixando o objeto em um estado incorreto. Em outras palavras, o resultado de várias transações concorrentes sendo executadas em um objeto deve ser igual ao resultado das mesmas transações executadas sequencialmente. Para isso se faz uso de técnicas de exclusão mútua, como *mutex locks*, semáforos binários e monitores.

Tópico 7 – Transparência de Falha:

A transparência de falha permite que usuários e programas completem as suas tarefas mesmo que uma falha ocorra em um dos nodos que disponibiliza um dos serviços sendo usados. A redundância de serviços entre nodos é um modo de garantir a transparência de falha, pois se um serviço é disponibilizado por mais de um nodo, caso um nodo deixe de funcionar, outros nodos que oferecem o mesmo serviço podem dar continuidade ao andamento das operações.

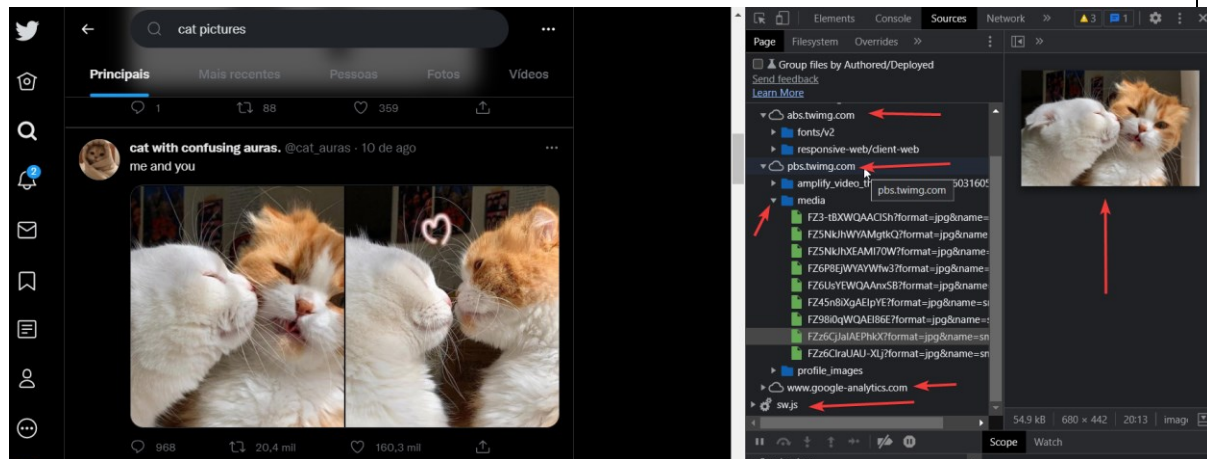
Atingir um nível máximo de transparência de falha é algo difícil pois muitas vezes é quase impossível saber a diferença entre um serviço que está completamente indisponível e entre um serviço que está muito lento. Além disso, muitas vezes um nível muito alto de transparência de falha não é desejado, pois pode piorar o desempenho aparente do sistema. Como exemplo, se o usuário contatar um servidor que está indisponível, o sistema automaticamente pode redirecionar sua requisição para outro servidor, que por sua vez também pode estar indisponível ou se tornar indisponível pouco tempo depois. Para o usuário, é plausível a ideia que ele está contatando somente um servidor que está muito lento.

Tarefa 2

Capture 10 telas de 10 diferentes produtos, sistemas, plataformas online, jogos, etc e cole aqui. Abaixo de cada uma das capturas de tela, explique por que o referido produto é de fato um sistema distribuído. Justifique os indícios e as características.

Você pode ser bem técnico. Veja meu exemplo:

Caso 1: Twitter



Este exemplo é do Twitter, provavelmente o microblog mais famoso que existe.

Nele, é possível perceber que o frontend é renderizado a partir de diversas fontes de conteúdo. As imagens que resultam da busca por “cat pictures” são servidas por um servidor, ou rede de servidores, que atende pelo nome “pbs.twimg.com”.

Outros servidores também são contatados para montar a tela da rede social. As fontes são servidas por abs.twimg.com, bem como possivelmente todos os artefatos web, html, Javascript e similares descarregados para o navegador chrome.

Dicas:

- Seu texto pode e deve ser mais longo que esse para detalhar sua explicação;
- O limite de 5 parágrafos é desejável;
- Há muitos SDs que não são páginas web. Lembre-se de trazê-los também.

Caso 2: BitTorrent

Ficheiros Info Pares Seguidores Graphs										
IP	Cliente	Opções	%	Vel. Recepção	Vel. Envio	Pedi...	Enviado	Recebido	Transf. Par	
213.24.132.113 [uTP]	MediaGet2 3.01.4223	D HP	100.0	835.5 KB/s	1.1 KB/s	167 0		11.0 MB		
213.138.195.87 [uTP]	µTorrent 3.5.5	D HP	100.0	528.5 KB/s	1.4 KB/s	142 0		9.50 MB		
pppoe.178-66-156-172.dynamic.avangarddsl.ru [uTP]	µTorrent 3.5.5	D HP	100.0	279.7 KB/s	0.7 KB/s	86 0		7.12 MB		
137-192-46-59.novotelecom.ru [uTP]	µTorrent 3.5.5	D HP	100.0	13.8 KB/s		66 0		3.51 MB		
217.73.192.236	µTorrent 3.5.5	D H	100.0	83.5 KB/s	0.3 KB/s	35 0		1.59 MB		

BitTorrent é um serviço de compartilhamento de arquivos de maneira distribuída.

Um modelo mais tradicional de download seria o de um servidor central que fornece um arquivo à varios clientes. Esse modelo sofre problemas de escalabilidade, visto que a taxa de transferência fica cada vez pior em razão da quantidade de clientes que se conectarem. BitTorrent soluciona esse problema através do download distribuído. Cada pessoa que baixa o conteúdo de um arquivo .torrent se torna um *seeder*, que é um distribuidor de partes desse arquivo para outras pessoas que baixarem o mesmo arquivo no futuro. Isso significa que o download de um arquivo por BitTorrent é feito através de vários *seeders*, onde cada um fornece uma parte diferente do arquivo, de maneira paralela. Quanto mais *seeders* melhor, já que então cada *seeder* irá fornecer uma parte cada vez menor do arquivo.

Essas características permitem que o processo de download seja descentralizado (evitando situações de gargalo que acontecem em servidores centralizados) e altamente paralelizado.

A imagem acima mostra 5 pares no uTorrent (uma aplicação cliente do BitTorrent). Pares (ou *peers*, em inglês) são outros hosts que também estão baixando ou já baixaram o mesmo arquivo que o usuário. Na coluna de “Recebido”, é possível ver a quantidade de dados já recebidas de cada par. Mesmo durante o processo de download, o BitTorrent faz *seed* das partes já baixadas.

Caso 3: DNS

```
[gustavomuller@fedoramuller ~]$ dig www.univali.br +trace

; <<>> DiG 9.16.29-RH <<>> www.univali.br +trace
;; global options: +cmd
.        64551    IN      NS     a.root-servers.net.
.        64551    IN      NS     b.root-servers.net.
.        64551    IN      NS     c.root-servers.net.
.        64551    IN      NS     d.root-servers.net.
.        64551    IN      NS     e.root-servers.net.
.        64551    IN      NS     f.root-servers.net.
.        64551    IN      NS     g.root-servers.net.
.        64551    IN      NS     h.root-servers.net.
.        64551    IN      NS     i.root-servers.net.
.        64551    IN      NS     j.root-servers.net.
.        64551    IN      NS     k.root-servers.net.
.        64551    IN      NS     l.root-servers.net.
.        64551    IN      NS     m.root-servers.net.
;; Received 239 bytes from 127.0.0.53#53(127.0.0.53) in 9 ms

br.      172800   IN      NS     a.dns.br.
br.      172800   IN      NS     b.dns.br.
br.      172800   IN      NS     c.dns.br.
br.      172800   IN      NS     d.dns.br.
br.      172800   IN      NS     e.dns.br.
br.      172800   IN      NS     f.dns.br.
br.      86400    IN      DS     2471 13 2 5E4F35998B8F909557F
br.      86400    IN      RRSIG  DS 8 1 86400 20221014050000 2
GP9B01yYQJKtm6CwPds0tAhvRPeyjOD uoB+pmcU80vek3AxcIT4+zCY/TFzj00B0YeSehoVhFLFq
;; Received 742 bytes from 198.97.190.53#53(h.root-servers.net) in 11 ms

univali.br. 3600    IN      NS     ns1.univali.br.
univali.br. 3600    IN      NS     ns2.univali.br.
univali.br. 900     IN      NSEC   univam.br. NS RRSIG NSEC
univali.br. 900     IN      RRSIG  NSEC 13 2 900 20221015121010
;; Received 268 bytes from 200.219.148.10#53(a.dns.br) in 10 ms

www.univali.br. 3600    IN      A      200.169.52.190
univali.br. 3600    IN      NS     ns1.univali.br.
univali.br. 3600    IN      NS     ns2.univali.br.
;; Received 155 bytes from 200.169.52.4#53(ns1.univali.br) in 7 ms

[gustavomuller@fedoramuller ~]$ _
```

DNS é o protocolo responsável por transformar nomes de domínio (como “www.univali.br”) em endereços IP. O sistema DNS é construído de maneira hierárquica e distribuída.

Na imagem acima, o utilitário *dig* é usado para resolver o nome de domínio “www.univali.br” em um endereço IP. A consulta é passada para um servidor *resolver* que então contata os 13 servidores raízes (letras “A” até “M”). O servidor raiz que responde à consulta é o servidor “H” (h.root-servers.net), enviando os endereços IP de vários servidores TLD (top level domain) responsáveis pelo domínio de topo “br”.

Após o servidor *resolver* receber essa resposta, ele decide contatar o servidor TLD chamado de “a.dns.br”. O servidor “a.dns.br” responde com os servidores autoritários do domínio “www.univali.br”, que são “ns1.univali.br” e “ns2.univali.br”.

Após o servidor *resolver* receber essa resposta, ele contata o servidor autoritário “ns1.univali.br” e recebe o resultado 200.169.52.190, que é o endereço IP mapeado ao nome de domínio “www.univali.br”.

As vantagens dessa distribuição incluem a redundância dos servidores raízes que oferecem tolerância a falhas, e também o maior desempenho, já que os servidores raízes são espalhados pelo mundo. Por causa disso, o servidor *resolver* pode escolher entre contatar um servidor raiz menos ocupado ou geograficamente mais próximo.

Um ponto interessante é que os servidores raízes, TLD e autoritários não oferecem transparência de replicação, já que os seus usuários possuem conhecimento da existência de diversas réplicas do mesmo serviço.

Caso 4: Netflix

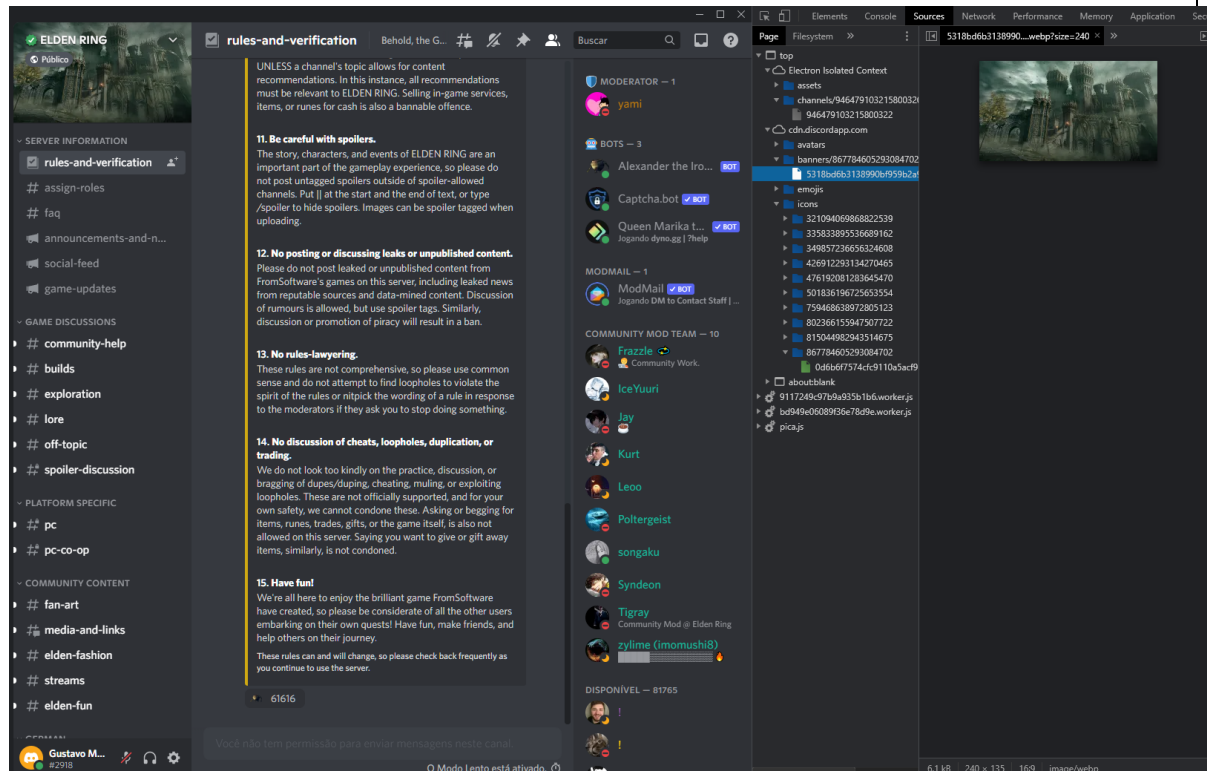
Status	Mét.	Domínio	Arquivo	Iniciador	Tipo	Transferido	Tam.
200	GET	ipv4-c001...	23481421-237438877o=10v=1130e=1664e	cadmium-pla...	octe...	256,77 KB	256...
200	GET	ipv4-c002...	138655817-139278757o=10v=390e=1664e	cadmium-pla...	octe...	606,84 KB	606...
200	GET	ipv4-c001...	23743888-240056411e=10v=1130e=1664e	cadmium-pla...	octe...	256,06 KB	255...
200	GET	ipv4-c003...	138276758-139550897o=10v=390e=1664e	cadmium-pla...	octe...	272,27 KB	271...
200	GET	ipv4-c003...	139555090-1399003667o=10v=390e=1664e	cadmium-pla...	octe...	345,45 KB	345...
200	GET	ipv4-c003...	139900367-1401871327o=10v=390e=1664e	cadmium-pla...	octe...	272,69 KB	272...
200	GET	ipv4-c003...	140187133-1406139497o=10v=390e=1664e	cadmium-pla...	octe...	417,27 KB	416...
200	GET	ipv4-c001...	24009642-242874357e=10v=1130e=1664e	cadmium-pla...	octe...	256,12 KB	255...
200	GET	ipv4-c003...	140613950-1408233967o=10v=390e=1664e	cadmium-pla...	octe...	205,00 KB	204...
200	GET	ipv4-c003...	140823397-1411227637o=10v=390e=1664e	cadmium-pla...	octe...	292,81 KB	292...
200	GET	ipv4-c003...	141122764-1417015597o=10v=390e=1664e	cadmium-pla...	octe...	565,69 KB	565...
200	GET	ipv4-c003...	141701560-1421467307o=10v=390e=1664e	cadmium-pla...	octe...	435,19 KB	434...
200	GET	ipv4-c001...	24267436-245398797o=10v=1130e=1664e	cadmium-pla...	octe...	256,75 KB	256...
200	GET	ipv4-c002...	142146731-1426973037o=10v=390e=1664e	cadmium-pla...	octe...	538,13 KB	537...
200	GET	ipv4-c003...	142697304-143170437o=10v=390e=1664e	cadmium-pla...	octe...	439,66 KB	439...
200	GET	ipv4-c003...	14317044-143643537o=10v=390e=1664e	cadmium-pla...	octe...	534,94 KB	534...
200	GET	ipv4-c001...	24529880-247916507o=10v=1130e=1664e	cadmium-pla...	octe...	256,09 KB	255...
200	GET	ipv4-c003...	143694354-1444304417o=10v=390e=1664e	cadmium-pla...	octe...	719,29 KB	718...
200	GET	ipv4-c003...	144430442-1446646387o=10v=390e=1664e	cadmium-pla...	octe...	229,17 KB	228...
200	GET	ipv4-c003...	144664639-1454436367o=10v=390e=1664e	cadmium-pla...	octe...	761,20 KB	760...
200	GET	ipv4-c001...	24791651-250534217o=10v=1130e=1664e	cadmium-pla...	octe...	256,09 KB	255...
200	GET	ipv4-c003...	145443637-1463986697o=10v=390e=1664e	cadmium-pla...	octe...	931,35 KB	930...
200	POST	www.netflix...	akiraClient.js...	akiraClient.js...	xmli	8,91 KB	0 B
200	POST	www.netflix...	akiraClient.js...	akiraClient.js...	xmli	9,46 KB	0 B
200	GET	ipv4-c003...	146398670-1466757947o=10v=390e=1664e	cadmium-pla...	octe...	272,84 KB	272...
200	POST	www.netflix...	akiraClient.js...	akiraClient.js...	xmli	11,16 KB	0 B
200	POST	www.netflix...	akiraClient.js...	akiraClient.js...	xmli	7,85 KB	0 B
200	POST	www.netflix...	1?reqAttempt=1&reqName=events/keepAl...	cadmium-pla...	json	4,99 KB	3,46...
200	POST	www.netflix...	1?reqAttempt=1&reqName=logblob/cleer...	cadmium-pla...	json	5,07 KB	3,48...
200	POST	www.netflix...	1?reqAttempt=1&reqName=events/keepAl...	cadmium-pla...	json	4,98 KB	3,46...
200	GET	ipv4-c003...	146675795-1468706387o=10v=390e=1664e	cadmium-pla...	octe...	190,74 KB	190...
200	GET	ipv4-c003...	146870639-1477139477o=10v=390e=1664e	cadmium-pla...	octe...	824,00 KB	823...
200	POST	www.netflix...	akiraClient.js...	akiraClient.js...	xmli	9,10 KB	0 B

Na imagem acima, é possível ver que ao assistir uma série ou filme na Netflix, os frames do conteúdo são recebidos por respostas HTTP vindas de domínios diferentes (“https://ipv4-c001-nvt002...”, “https://ipv4-c002-nvt002...” e “https://ipv4-c003-nvt002...”). O tamanho das respostas não passa de 500KB e o conteúdo de cada uma é uma string codificada em base 64. Logo, o *streaming* de vídeos acontece de maneira distribuída, onde cada nodo do sistema fornece uma parte diferente do vídeo de maneira paralela, de modo similar ao BitTorrent.

Outro ponto que indica que a Netflix é um sistema distribuído é a tecnologia Open Connect, que serve como um cache de vídeos e filmes que é oferecido para provedoras de internet grandes, com

o objetivo de diminuir a latência, pois aproxima geograficamente o conteúdo dos usuários. Esse é um exemplo de replicação.

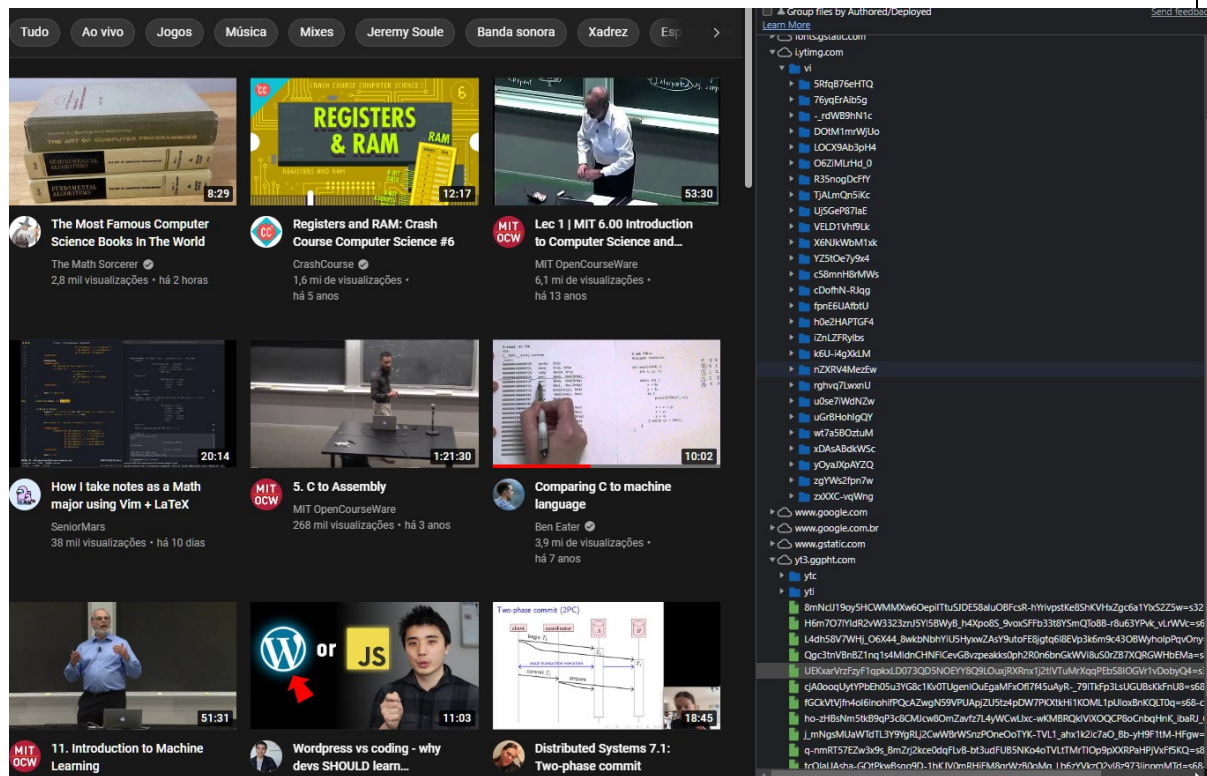
Caso 5: Discord



Na imagem acima é possível ver que a fonte do conteúdo presente em um servidor do Discord (como ícones, fotos de perfil e também imagens e vídeos enviados por usuários no chat) são disponibilizados por um host de nome “cdn.discordapp.com”. O prefixo “cdn” significa *Content Delivery Network*, que é um sistema de computadores que fornecem conteúdo à uma quantidade massiva de usuários de maneira rápida e transparente. Por definição, CDNs são sistemas distribuídos que fazem uso de redundância de dados para diminuir a distância entre o usuário e o conteúdo, diminuindo a latência.

Em contraste ao exemplo passado da Netflix, todas as imagens aparentam ter o mesmo domínio como origem (cdn.discordapp.com), mas isso acontece devido à transparência de localização e replicação. Logo, como usuário, não é possível saber de qual lugar uma imagem veio especificamente, e também não é possível saber se existe mais de uma cópia dela ou não.

Caso 6: Youtube



Na imagem acima, ao carregar a página inicial do Youtube, vários servidores diferentes são contatados para que página possa ser “montada”. Por exemplo, o domínio “i.ytimg.com” é contatado para obter as thumbnails, os domínios “google.com”, “google.com.br” e “gstatic.com” são contatados para obter alguns arquivos Javascript e as fontes usadas, e por último o domínio “yt3.ggpht.com” é contatado para obter as fotos de perfil dos canais.

Caso 7:

[imagem]

[descrição]

Caso 8

[imagem]

[descrição]

Caso 9

[imagem]

[descrição]

Caso 10

[imagem]

[descrição]

Tarefa 3

Conforme discutido no início da disciplina, o assunto de sistemas distribuídos integra diversas matérias já vistas em ciência da computação ou mesmo em engenharia de computação e sistemas para internet. Nesta tarefa solicito que escreva um texto de aproximadamente 500 palavras sobre como sistemas distribuídos se relaciona com as seguintes disciplinas que servem de requisito para nosso estudo.

A palavra relação indica que você pode comparar, contrastar, definir, integrar e relacionar o tema de sistemas distribuídos com sua contraparte.

Sistemas Distribuídos e Redes de Computadores (500+ palavras)

Como um sistema distribuído é um conjunto de computadores independentes que possuem um objetivo em comum, esses computadores precisam se comunicar e trocar informações de um modo (normalmente) seguro, garantido e confiável. Esse é exatamente o problema geral estudado em Redes de Computadores. A própria internet pode ser considerada um sistema distribuído.

Algumas das vantagens de sistemas distribuídos usarem uma rede de computadores para a comunicação são:

- Maior proteção contra falhas. Como exemplo, caso parte de uma rede pare de funcionar, as outras partes do sistema que estão em partes diferentes da rede podem continuar à funcionar, caso não dependam da parte indisponível;
- Redução de congestionamento de pacotes na rede, já que o fluxo de rede não vai estar focado em somente um ponto;
- Maior desempenho e confiabilidade ao usar réplicas de serviços posicionadas em diferentes pontos geográficos;

Algumas das desvantagens de sistemas distribuídos usarem uma rede de computadores para a comunicação são:

- O fato dos nodos de um sistema distribuído se comunicarem através de uma rede adiciona pontos de falha diferentes no sistema, já que a rede em si também é suscetível à falhas;
- O envio de mensagens pela rede normalmente implica em pior desempenho por causa que a mensagem da camada de aplicação é encapsulada, desencapsulada e redirecionada múltiplas vezes por vários nodos que estão entre a origem e o destino. Por mais que um sistema monolítico também use redes de computadores (como um conjunto de mainframes conectados por Ethernet), normalmente uma pilha de protocolos menor é

usada, reduzindo a quantidade de operações necessárias para que uma mensagem vá da origem até o destino;

- Segurança, pois há mais elementos que devem ser protegidos;
- A falta de noção de um único relógio global em sistemas distribuídos (já que cada nodo possui o seu próprio relógio local) acarreta na geração de vários problemas de sincronismo.
- Por último, como um sistema monolítico é centralizado e “isolado” do resto do mundo, a conexão entre as suas parte internas não precisa necessariamente fazer uso das estruturas de conexão entre hosts já existentes no mundo (como a internet), logo, é mais fácil implementar protocolos de intra-comunicação customizados. Em sistemas distribuídos isso não é verdade. Por exemplo, se um sistema distribuído fosse espalhado por toda a américa do norte, américa central e américa do sul, é muito provável que os seus nodos usariam os protocolos de comunicação já existentes (como IP, TCP, UDP, Ethernet, etc...) para se comunicarem, já que seria inviável criar uma estrutura nova e customizada que tivesse um alcance continental.

Sistemas Distribuídos e Sistemas Operacionais (500+ palavras)

A relação entre sistemas distribuídos e sistemas operacionais é que o sistema operacional faz parte da arquitetura de um sistema distribuído. Como cada nodo pode ter um sistema operacional diferente e isso causa problemas de compatibilidade, é comum que sistema distribuído tenha uma camada adicional de *middleware*, logo acima da camada do sistema operacional, que tem como objetivo esconder as especificidades de cada sistema operacional, fornecendo uma interface padronizada. Essa interface padronizada seria o método de assegurar a transparência de acesso.

O middleware normalmente é um conjunto de bibliotecas e ferramentas que podem ser usadas em linguagens de programação comuns. Uma alternativa é o uso de linguagens de programação construídas especificamente para a computação distribuída (como Erlang, Ada, Limbo, etc.). A funcionalidade mais importante oferecida é o envio e recebimento de mensagens de um nodo para outro, que forma a base de como os nodos de um sistema distribuído se comunicam.

Outra opção é o uso de um sistema operacional especificamente criado para sistemas distribuídos (DOS – Distributed Operating System). Nesse caso, o *middleware* talvez não seria necessário. Sistemas operacionais distribuídos geralmente assumem que os nodos são homogêneos, mas recentemente o interesse pela arquitetura de *middleware* vem crescendo devido a maior flexibilidade, já que as diferenças entre os nodos é abstraída pelo *middleware*.

Sistemas Distribuídos e Programação (500+ palavras)

A programação de sistemas distribuídos é naturalmente paralela, visto que cada nodo possui a sua própria noção de relógio e sua própria linha de execução isolada. Porém, a natureza distribuída permite a criação de novos algoritmos que solucionam problemas de maneiras diferentes e muitas vezes mais eficientes. Um exemplo disso é o roteamento por Distance Vector, onde inicialmente cada nodo propaga para o resto da rede o custo imediato de todos os seus enlaces adjacentes e a cada iteração seguinte cada nodo é responsável por atualizar a sua própria tabela de roteamento com base nos custos imediatos recebidos de outros nodos. A computação é realizada de maneira descentralizada, em contraste à outros métodos, como um roteamento centralizado que é calculado por um controlador SDN.

Por causa dessa natureza paralela, a sincronização e coordenação entre nodos de um sistema distribuído é um problema fundamental da área. É possível também, de maneira abstrata, visualizar cada nodo de um sistema distribuído como sendo uma “thread” diferente.

Um sistema distribuído é um sistema multicomputador. Isso significa que a memória de cada unidade de processamento é privada, em contraste à sistemas multiprocessadores, onde a memória é compartilhada. A vantagem do modelo multicomputador é que não é necessário métodos de exclusão mútua para acesso à memória entre computadores, já que o uso da memória é privado. Por outro lado, o compartilhamento de memória tende à ser mais rápido em sistemas onde processadores diferentes habitam o mesmo computador e possuem acesso direto (público) à memória.